

1. What is Docker?

Docker is a platform that allows you to automate the deployment, scaling, and management of applications in lightweight containers. Containers are isolated environments that package an application and all its dependencies, ensuring it runs consistently across different computing environments.

Key Concepts

- **Image:** A lightweight, standalone, and executable software package that includes everything needed to run a piece of software, including the code, runtime, libraries, and dependencies.
- **Container:** A runnable instance of an image. You can think of it as a lightweight, standalone, and executable package of software.
- **Dockerfile:** A script that contains instructions on how to build a Docker image.
- **Docker Hub:** A public registry that contains a large number of Docker images shared by the community.

2. Setting Up Docker

1. **Installation:**
Download and install Docker Desktop from the official Docker website.
2. **Basic Commands:**
 - o `docker --version`: Check the Docker version installed.
 - o `docker ps`: List running containers.
 - o `docker images`: List downloaded images.
 - o `docker build`: Build an image from a Dockerfile.
 - o `docker run`: Run a container from an image.

3. Creating a Java Spring Boot Application

Let's create a simple Spring Boot application and then containerize it using Docker.

Step 1: Create a Spring Boot Project

You can use [Spring Initializr](#) to bootstrap your project:

1. **Project Metadata:**
 - o Project: Maven Project
 - o Language: Java
 - o Spring Boot: 3.0.0 (or latest stable version)
 - o Group: com.example
 - o Artifact: demo
 - o Name: demo
 - o Packaging: Jar
 - o Java: 17 (or your preferred version)
2. **Dependencies:**
 - o Spring Web

Download the generated project and unzip it.

Step 2: Writing a Simple Controller

Create a simple REST controller in your application:

```
java
Copy code
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/")
    public String hello() {
        return "Hello, Docker!";
    }
}
```

Step 3: Build the Application

Build the application using Maven:

```
bash
Copy code
./mvnw clean package
```

This will create a JAR file in the target directory.

4. Dockerizing the Spring Boot Application

Step 1: Create a Dockerfile

In the root of your project directory, create a file named Dockerfile with the following content:

```
dockerfile
Copy code
# Start with a base image containing Java runtime
FROM openjdk:17-jdk-alpine

# Add Maintainer info
LABEL maintainer="your-email@example.com"

# Add a volume pointing to /tmp
VOLUME /tmp

# Make port 8080 available to the world outside this container
EXPOSE 8080

# The application's jar file
ARG JAR_FILE=target/demo-0.0.1-SNAPSHOT.jar

# Add the application's jar to the container
ADD ${JAR_FILE} app.jar

# Run the jar file
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Step 2: Build the Docker Image

Navigate to your project directory and build the Docker image using the following command:

```
bash
Copy code
docker build -t spring-boot-demo .
```

This command will create an image named `spring-boot-demo` based on the instructions in your Dockerfile.

Step 3: Run the Docker Container

To run the application inside a Docker container, use the following command:

```
bash
Copy code
docker run -p 8080:8080 spring-boot-demo
```

This command will run the container, mapping the container's port 8080 to your local machine's port 8080. You can access the application at `http://localhost:8080`.

5. Key Docker Commands for Managing Containers

- **List running containers:**

```
bash
Copy code
docker ps
```

- **Stop a container:**

```
bash
Copy code
docker stop <container_id>
```

- **Remove a container:**

```
bash
Copy code
docker rm <container_id>
```

- **Remove an image:**

```
bash
Copy code
docker rmi <image_id>
```

6. Next Steps

Now that you've containerized a simple Spring Boot application, you can explore more advanced topics like:

- **Docker Compose:** Orchestrating multi-container applications.

- **Docker Networking:** Understanding Docker's networking model.
- **Docker Volumes:** Persisting data in Docker.

Docker Image automation with single Maven command

- Spring boot application
- Add Dockerfile
 1. FROM openjdk
 2. EXPOSE 8080
 3. ADD target/demo-0.0.1-SNAPSHOT.jar
 4. ENTRYPOINT ["java","-jar","/app.jar"]

Add tag <finalname>DockerSpringbootDemo</finalname>

Auto Push Spring Boot App Image on Docker Hub Repository#9561826040

Pom.xml plugin

<plugin>

<groupId>com.spotify</groupId>

<artifactId>dockerfile-maven-plugin</artifactId>

<version>1.4.13</version>

<executions>

<execution>

<id>default</id>

<goals>

<goal>build</goal>

<goal>push</goal>

</goals>

</execution>

</executions>

<configuration>

```
<repository>gmantrig/bootapp1</repository>
```

```
<tag>11.8</tag>
```

```
<useMavenSettingsForAuth>true</useMavenSettingsForAuth>
```

```
</configuration>
```

```
</plugin>
```

```
</plugins>
```

```
=====
=====
=====
```

settings.xml m2 folder dockerhiub username password

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0
  http://maven.apache.org/xsd/settings-1.0.0.xsd">
```

```
  <servers>
```

```
    <server>
```

```
      <id>docker.io</id>
```

```
      <username>gmantrig</username>
```

```
      <password>Perfect@12345</password>
```

```
    </server>
```

```
  </servers>
```

```
  <pluginGroups>
```

```
    <pluginGroup>com.spotify</pluginGroup>
```

```
  </pluginGroups>
```

</settings>

1. What is Docker?

Answer:

Docker is a platform that makes it easy to create, deploy, and run applications in containers. Containers are lightweight and portable environments that include everything needed to run an application, ensuring it works the same way regardless of where it's deployed.

2. What is a Docker container?

Answer:

A Docker container is a lightweight, standalone, and executable package that includes an application and all its dependencies. Containers run the same regardless of the environment, providing consistency across different development, testing, and production environments.

3. What is a Docker image?

Answer:

A Docker image is a read-only template that contains the instructions for creating a Docker container. It includes everything needed to run a piece of software, such as the code, runtime, libraries, and configurations.

4. What is a Dockerfile?

Answer:

A Dockerfile is a script containing a set of instructions to build a Docker image. It specifies the base image, the application code, and any other dependencies required for the application.

5. What is Docker Hub?

Answer:

Docker Hub is a cloud-based repository where Docker users and partners create, test, store, and distribute container images. It is like a public library for Docker images, where you can find and share images.

6. What is the difference between a Docker container and a virtual machine (VM)?

Answer:

- **Docker Container:** Lightweight, shares the host system's kernel, starts quickly, and uses less system resources.
- **Virtual Machine:** Heavier, includes a full operating system, starts slowly, and uses more system resources.

7. How do you create a Docker container?

Answer:

You create a Docker container by running an image using the command:

```
bash
```

Copy code
docker run <image_name>

For example, to run a container from an image called nginx, you would use:

bash
Copy code
docker run nginx

8. How do you list all running Docker containers?

Answer:

You can list all running Docker containers with the command:

bash
Copy code
docker ps

9. How do you stop a running Docker container?

Answer:

You can stop a running Docker container using the command:

bash
Copy code
docker stop <container_id>

You can find the container_id by using the docker ps command.

10. How do you delete a Docker container?

Answer:

You can delete a Docker container using the command:

bash
Copy code
docker rm <container_id>

11. How do you delete a Docker image?

Answer:

You can delete a Docker image using the command:

bash
Copy code
docker rmi <image_id>

12. What is Docker Compose?

Answer:

Docker Compose is a tool that allows you to define and run multi-container Docker applications. You can define your application services, networks, and volumes in a single docker-compose.yml file, making it easy to start and stop all services with a single command.

13. What is the purpose of the EXPOSE instruction in a Dockerfile?

Answer:

The EXPOSE instruction in a Dockerfile indicates the ports on which a container will listen for connections. It doesn't actually publish the ports; it just serves as documentation and a hint for anyone running the container.

14. What is a Docker Volume?**Answer:**

A Docker Volume is a storage mechanism for persisting data generated by and used by Docker containers. Volumes are managed by Docker and can be used to share data between containers or between a container and the host system.

15. What is the difference between CMD and ENTRYPOINT in a Dockerfile?**Answer:**

- **CMD:** Specifies the default command to run when a container starts. It can be overridden by specifying a command in docker run.
- **ENTRYPOINT:** Specifies the command that will always run when the container starts. You can still pass arguments to it.